République Tunisienne

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

École Supérieure Privée d'ingénierie et de technologie

TEK-UP

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques

Spécialité : Web, Mobile and Multimedia Development

Réalisé par

Nadim JEBALI

Design and Development of an Intelligent Multi-Vendor Marketplace: Cheebo

Professional Supervisor: Mr. Mohammed Aziz Chibani
NeuralBey

Academic Supervisor: Ms. Sawssen Jalel
Lecturer

Academic Year 2025 – 2026

I authorise the student to submit his internship report for a defence.

Professional Supervisor, Mr. Mohammed Aziz Chibani

Signature and stamp

I authorise the student to submit his internship report for a defence.

Academic Supervisor, Ms. Sawssen Jalel

Signature

Dedication

To my parents, for their unwavering love, patience and encouragement during every step of this project.

To my friends and colleagues at NeuralBey and TEK-UP who offered guidance, feedback and inspiration.

Acknowledgements

I would like to express my sincere gratitude to all those who contributed to the completion of this project.

First, my deepest thanks to my professional supervisor, Mr. Mohammed Aziz Chibani (NeuralBey), for his continuous guidance, domain expertise, and support throughout the development and evaluation phases.

I am also grateful to Ms. Sawssen Jalel for her academic supervision and valuable feedback which helped shape the report.

Special thanks to my colleagues and friends for testing features, providing constructive critiques, and motivating me during challenging moments.

Finally, I thank my family for their encouragement and patience during this work.

Abstract

Keywords: marketplace, multi-vendor, NestJS, React, Prisma, Artificial Intelligence, Docker, CI/CD, Terraform.

Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CD	Continuous Deployment
CI	Continuous Integration
DB	Database
DNS	Domain Name System
DTO	Data Transfer Object
DWMM	Web, Mobile and Multimedia Development
JWT	JSON Web Token
ORM	Object-Relational Mapping
PFE	Final Year Project
REST	Representational State Transfer
SPA	Single Page Application
SSH	Secure Shell
UML	Unified Modeling Language
VPC	Virtual Private Cloud

General Introduction

This introduction explains the context for the Cheebo project, why the system was created, who the intended users are, and how the work was organised from conception through delivery.

Cheebo is a multi-vendor marketplace developed as a final year engineering project in partnership with TEK-UP and NeuralBey. The platform is designed to give Tunisian sellers a modern, secure online storefront while offering customers an intelligent shopping experience and administrators a reliable way to manage vendors, products, and verification workflows.

The project combines academic goals and real-world needs: it must satisfy the PFE requirements for software engineering, while also providing a practical solution that addresses the host organisation's expectations for a modern web application. The following chapters present the host organisation, the business context, the requirements analysis, the system architecture, implementation details, and the deployment strategy.

Chapter 1 General Project Context

This chapter sets the scene — why this project exists, who it is for, and how its development was organised. It introduces the host organisation, the business motivation behind Cheebo, the competitive landscape in which it operates, the proposed solution, and the working methodology adopted throughout the internship.

1.1 Presentation of the Host Organisation

1.1.1 NeuralBey — Company Overview

NeuralBey is a Tunisian technology company that delivers end-to-end engineering services across a broad spectrum of modern computing fields. The company positions itself as a partner that translates client visions into working systems, summarised by its own statement: *"From web and mobile apps to AI systems, IoT, embedded engineering, and 3D solutions — we build the technology that powers your vision."*

This breadth allows NeuralBey to handle a project from initial concept through deployment without external dependencies, and to combine disciplines that are usually siloed — for example, integrating artificial intelligence into conventional web platforms, or interfacing embedded devices with cloud services. The company's portfolio reflects this versatility:

- **Web and mobile applications** for consumer products and enterprise tooling;
- **AI systems**, including model integration, automation pipelines and intelligent assistants;
- **IoT and embedded engineering**, where NeuralBey designs firmware and connected devices;
- **3D solutions** for product visualisation, AR/VR and digital twins.

1.1.2 Domain, Team and Mission

NeuralBey operates as a multidisciplinary studio. Each project is staffed by engineers selected from the relevant domain (frontend, backend, AI, hardware, 3D), and a professional supervisor accompanies the work from scoping through delivery. The team's working culture emphasises code review, iterative delivery and frequent stakeholder

feedback — practices that aligned naturally with the academic requirements of a final year project.

The present project was carried out under the professional supervision of **Mr. Mohammed Aziz Chibani**, who provided architectural guidance, regular reviews, and technology recommendations throughout the internship. Academic oversight was provided in parallel by **Ms. Sawssen Jalel**, ensuring the work also satisfied the methodological and documentation standards of the engineering diploma.

1.2 Project Context

1.2.1 Project Origin and Motivation

The Cheebo project originated from an internal need identified at NeuralBey: the Tunisian online retail landscape lacks a modern, multi-vendor platform that is both seller-friendly and equipped with the automation features expected of contemporary marketplaces. Existing local solutions are typically single-vendor stores or classified-ad sites, and they offer limited tooling for moderation, content quality and discovery.

NeuralBey saw an opportunity for a product combining three differentiating capabilities:

(i) a true multi-vendor model with self-service onboarding for sellers;

(ii) AI-assisted moderation of stores and products to reduce manual review load;

(iii) a deployment story that demonstrates contemporary DevOps practices end-to-end.

The PFE topic was therefore scoped jointly with the professional supervisor, with the explicit goal of producing a system that could serve both as a portfolio piece for the company and as a foundation for further internal development beyond the internship.

1.2.2 Multi-Vendor Marketplace Concept

A *multi-vendor marketplace* is a platform on which independent sellers operate their own stores within a shared environment governed by a central operator. It differs from two adjacent models:

- a **single-vendor store**, where the platform itself owns the inventory and the brand;
- a **classified-ad site**, where the platform only lists offers and plays no role in transactions or quality control.

A marketplace must reconcile three concerns simultaneously: **seller autonomy** (each seller manages catalogue, branding and inventory), **buyer trust** (the platform

guarantees a baseline of product authenticity and fulfilment) and **operator governance** (administrators onboard sellers, moderate content, suspend stores and curate featured selections). These three perspectives drive the architecture of Cheebo: every feature is designed to balance autonomy for sellers, trust for buyers, and control for administrators. The integration of AI verification is a direct response to the buyer-trust requirement at the scale a marketplace demands.

1.2.3 Target Users

Three actor categories were identified at the scoping stage:

- **Customer** — an end-user who browses the catalogue, places orders and tracks deliveries. Expects a fast, mobile-friendly experience with reliable search and clear product information.
- **Seller** — a small or medium merchant who creates a store, manages a product catalogue and fulfils orders. Expects low onboarding friction, clear analytics, and protection against unfair competition from low-quality listings.
- **Administrator** — the platform operator (NeuralBey staff) who reviews seller applications, moderates flagged content, curates featured stores, and intervenes when a store violates the marketplace rules.

1.3 Study of Existing Solutions

1.3.1 Existing Tunisian E-Commerce Platforms

Three categories of competing platforms were studied:

- **Pan-African marketplaces (Jumia).** Jumia Tunisia is the most visible regional marketplace. It supports multiple vendors at scale and offers a polished customer experience, but seller onboarding is heavyweight and the platform is not tailored to small Tunisian merchants.
- **Classified-ad sites (Tayara, Affariyet).** Tayara dominates the local classified-ad market. It offers excellent discovery for sellers, but performs no escrow, no order management, no integrated payment and no quality moderation.
- **Single-vendor stores (MyTek, Wiki, etc.).** These are vertically integrated retailers that operate their own catalogue. They cannot be used by independent sellers.

1.3.2 Limitations of Current Solutions

From the study above, four gaps emerged:

L1. No AI-assisted moderation. On existing platforms, store and product verification are entirely manual — slow, inconsistent, and impractical at scale.

L2. Heavyweight seller onboarding. Pan-regional players require contractual paperwork unsuitable for small merchants.

L3. Limited operator tooling. Classified-ad sites have weak admin dashboards; single-vendor stores have none for external sellers.

L4. Outdated technical foundations. Several platforms still rely on monolithic stacks without modern CI/CD or infrastructure-as-code, making evolution costly.

1.3.3 Competitive Analysis

Table 1.1 summarises the comparison along the four criteria that drove the design of Cheebo. A check mark (✓) indicates the criterion is fully supported, a cross (✗) indicates it is absent, and a tilde (∼) indicates partial support.

Table 1.1: Comparative analysis of competing platforms

Criterion	Jumia	Tayara	Cheebo
Multi-vendor model	✓	∼	✓
AI verification	✗	✗	✓
Admin dashboard	∼	✗	✓
CI/CD pipeline (open)	✗	✗	✓
Lightweight onboarding	✗	✓	✓

The `C` column type used above is defined locally as a centred variant of `X`; it is declared once in the preamble.

1.4 Proposed Solution

1.4.1 Cheebo Overview

Cheebo is a multi-vendor marketplace designed around three pillars: a self-service seller experience, AI-assisted content verification, and a modern, fully containerised deployment. The platform is built as a single-page **React 19** application backed by a modular **NestJS** API, with **MariaDB** as the relational store and **n8n** acting as an orchestration layer for AI tasks. Authentication is handled by **Better-Auth** with email verification, and **Prisma** is used as the ORM. The full system is packaged with Docker Compose, provisioned on DigitalOcean via Terraform, and delivered through a GitHub Actions CI/CD pipeline.

Sellers create a store in minutes, populate it with products and tags, and benefit from automatic AI verification that surfaces a trust badge to potential buyers. Administrators review borderline cases through a dedicated dashboard, suspend stores when needed, and override automated decisions. Customers browse a unified catalogue, place orders across multiple sellers in a single checkout, and interact with an in-app AI assistant for discovery and support.

1.4.2 Key Differentiators

Five differentiators set Cheebo apart from the platforms surveyed in the previous section:

D1. AI verification of stores and products via an n8n workflow that calls a hosted LLM and writes back an `adminReviewed` flag plus a textual rationale.

D2. Lightweight seller onboarding through a seller application form reviewed by administrators, with a free tier offering a single store at zero cost.

D3. Integrated administration tooling (review, suspend, curate, cascade-delete) implemented as first-class features rather than database admin scripts.

D4. Conversational data access via a database-aware chatbot built on n8n, allowing administrators to query the platform in natural language.

D5. Modern DevOps foundations — pnpm monorepo, Docker Compose, Terraform on DigitalOcean, and GitHub Actions CI/CD — demonstrating production-grade engineering practices end-to-end.

1.4.3 Project Scope

The scope was agreed with the professional supervisor and is split between in-scope and out-of-scope items so that the deliverables remain clear and bounded.

In scope: account creation and email verification; seller application workflow with admin review; store and product CRUD with image upload; tag-based product recommendations with a top-seller fallback; order placement, items and history; admin moderation panels (store suspension, AI-verified badges, cascade-delete); AI verification and chat assistant via n8n; premium tier unlocking additional stores; and infrastructure-as-code deployment.

Out of scope (deferred to future iterations): integrated payment processing (e.g. Konnect or Stripe); a native mobile application; advanced fraud detection; on-premise LLM hosting; multi-currency support; and a full-text search engine such as Elasticsearch or Typesense.

1.5 Work Methodology

1.5.1 Agile / Scrum Methodology

The development followed a **Scrum-inspired Agile methodology**, adapted to the constraints of a single-developer PFE. The student acted simultaneously as Product Owner — translating the NeuralBey brief into a backlog of user stories — and as the sole development team. Mr. Mohammed Aziz Chibani fulfilled the stakeholder role and combined the responsibilities normally split between Scrum Master and Product Owner: he approved the sprint backlog, conducted sprint reviews, and provided architectural and technological direction.

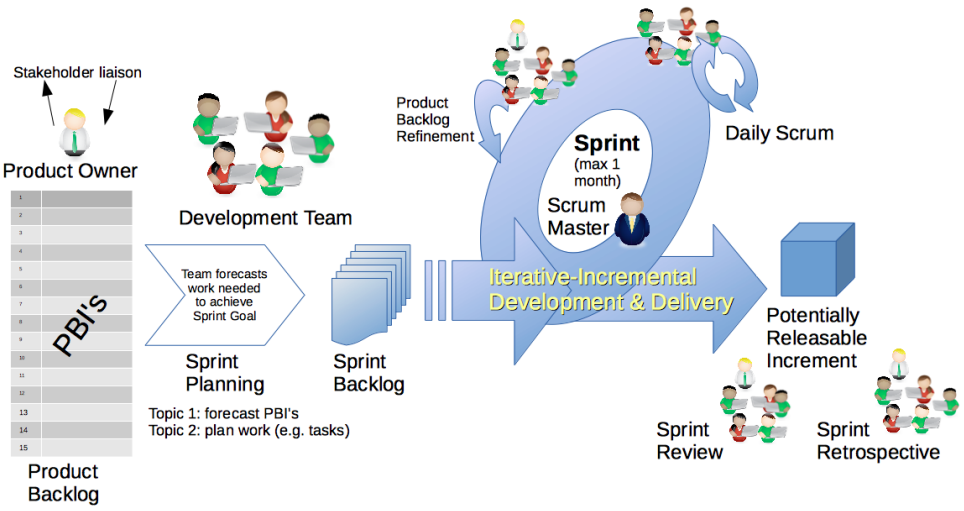

Figure 1.1: Scrum methodological framework (source: Dr Ian Mitchell, Wikimedia Commons, CC0 / CC BY-SA 4.0).

1.5.2 Sprint Planning

Work was organised in iterations of **two weeks**. Each sprint opened with a planning session in which the backlog items selected from the previous review were broken into technical tasks, and closed with a review meeting at NeuralBey where the working increment was demonstrated to Mr. Chibani. The review fed back into a revised backlog for the following sprint. A retrospective followed every second review to consolidate process improvements — for example, the adoption of pnpm for monorepo management, the migration from manual deployments to GitHub Actions, and the introduction of n8n for AI workflows.

The full project ran across approximately ten sprints, grouped into three high-level milestones:

- **Foundations (sprints 1–3):** repository setup, authentication with Better-Auth, base data model, and the seller-application flow.

- **Core marketplace (sprints 4–7):** stores, products, orders, admin panels, and tag-based recommendations.
- **Intelligence and delivery (sprints 8–10):** the AI verification pipeline, the in-app AI chat assistant, Dockerisation, Terraform provisioning, and the CI/CD pipeline.

1.5.3 Tools Used

The toolchain was kept deliberately minimal and standard. Table 1.2 lists the principal tools used through the project and their purpose.

Table 1.2: Project management and development tools

Tool	Purpose
GitHub	Source control, code review, and CI/CD via GitHub Actions
Notion	Backlog management, sprint board, and review notes
VSCode	Primary IDE with TypeScript, Prisma and ESLint extensions
Docker	Containerisation of the database, backend, frontend and n8n services
Terraform	Infrastructure-as-code provisioning on DigitalOcean
n8n	Visual orchestration of AI verification and chat workflows
Postman	Manual exercising of REST API endpoints during backend development
pnpm	Workspace and dependency management across the monorepo

GitHub centralised both source control and the CI/CD pipeline, removing the need for a separate runner host. Notion served as a lightweight backlog and sprint board, capturing user stories, acceptance criteria and review notes. Docker isolated runtime dependencies and ensured parity between the development environment and the production droplet. The eventual production deployment was shaped through Terraform and Docker Compose, with all secrets managed as GitHub Actions environment variables.

Key Takeaways

- Cheebo originates from an internal NeuralBey brief: build a modern, Tunisia-oriented multi-vendor marketplace with AI-assisted moderation.
- Three actors — customer, seller, administrator — drive every architectural decision throughout the report.
- Compared with Jumia and Tayara, Cheebo differentiates on *AI verification*, *lightweight seller onboarding*, *first-class admin tooling*, and *modern DevOps* (Docker, Terraform, GitHub Actions).

- Development followed two-week Scrum sprints with NeuralBey reviews, structured around three milestones: foundations, core marketplace, and intelligence-and-delivery.

Chapter 2 Requirements Analysis and Specification

This chapter identifies the three system actors, formalises the functional and non-functional requirements of Cheebo, and illustrates them with UML use-case and sequence diagrams. The requirements were consolidated during the first three sprints in collaboration with the professional supervisor and serve as the contract that the rest of the report builds upon.

2.1 Identification of Actors

The platform recognises three actor categories. They are distinct *roles* stored on the `User` table rather than distinct user records — a single account can be promoted from *customer* to *seller* once a seller application is approved, and an administrator account is created automatically on first boot.

2.1.1 Customer

The Customer is the default role assigned upon registration. A Customer can browse the catalogue, search products, view individual stores, manage a cart, place an order across multiple sellers in a single checkout, and review their personal order history. The Customer is also the entry point for the seller pipeline: any Customer can submit a seller application and, once approved, gain the Seller role on the same account. Anonymous (non-authenticated) visitors share the read-only abilities of the Customer but cannot maintain a cart or place an order.

2.1.2 Seller

The Seller is a Customer whose seller application has been approved by an administrator. A Seller can create and edit one store under the free tier (or several under the premium tier), populate that store with products, upload product images, attach product tags, view per-store analytics, and consult the order history specific to their store. Sellers also choose a visual template for their store from the templates exposed by the frontend. They cannot, however, review or moderate the work of other sellers.

2.1.3 Administrator

The Administrator is created automatically by the backend at first boot and is not exposed through the registration form. The Administrator reviews seller applications, browses every store and every product on the platform, suspends or deletes stores (with cascading deletion of dependent records), overrides automated AI verification decisions, curates featured stores and products on the landing page, manages categories and subcategories, and consults the platform through a database-aware AI chat assistant.

2.2 Functional Requirements

The functional requirements are grouped into seven domains. Each domain corresponds to a NestJS module on the backend and to a matching set of pages on the React frontend; the mapping is made explicit in Chapter 3.

2.2.1 User Registration and Authentication

FR-A1. The system shall allow a visitor to create an account with an email address and password.

FR-A2. The system shall send a verification email containing a one-time link upon successful registration.

FR-A3. The system shall prevent unverified accounts from creating a store or placing an order.

FR-A4. The system shall authenticate returning users via Better-Auth sessions and issue an HTTP-only cookie.

FR-A5. The system shall allow the user to log out and invalidate the corresponding session.

FR-A6. The system shall enforce role-based access control on every non-public route through guards and the `@Roles()` decorator.

2.2.2 Store Management

FR-S1. A Seller shall be able to create a store with a name, description, logo and template.

FR-S2. A Seller shall be able to edit their store profile at any time.

FR-S3. A free-tier Seller shall be limited to a single store.

FR-S4. A premium-tier Seller shall be able to operate multiple stores from the same account.

FR-S5. Each store shall expose a public storefront URL identified by a human-readable slug derived from the store name (e.g. `/stores/pawsome-pet-foods`), rather than by its numeric primary key.

FR-S6. Slug uniqueness shall be enforced at the database level and collisions resolved by an automatic numeric suffix (`-2`, `-3`, ...). The slug shall be regenerated when the store name changes.

FR-S7. The store status (active, pending, suspended) shall be persisted in a `StoreStatusLog` audit trail.

FR-S8. The Seller shall be able to customise the storefront across six independent sections — *Theme*, *Header*, *Banner*, *Sidebar*, *Product grid* and *Footer* — through a dedicated split-screen editor with a live preview pane.

FR-S9. Four pre-built presets shall be available as starting points for customisation: *Default* and *Minimal* are available to all sellers, while *Modern* and *Classic* are restricted to premium accounts. Applying a preset overwrites the current draft with the preset's section defaults.

FR-S10. The Seller shall be able to upload a custom banner image, stored under `/uploads/stores/` and referenced from the customisation JSON.

FR-S11. The customisation editor shall support undo/redo on every section change and shall warn the seller before discarding unsaved changes.

FR-S12. Customisation features shall be tiered. Free accounts may use a curated subset — a six-colour palette for the primary colour, two fonts (*Inter* and the system default), the *soft* corner radius, checkbox-style filters, square product images, and section visibility toggles. Premium accounts unlock the full set: free-form hex colours, all four fonts, all radii, banner image upload with heading and overlay controls, sidebar inner panel and text colours, non-checkbox filter styles, product grid background, non-square aspect ratios, and footer social links. When a premium subscription expires, the seller's existing customisation values are preserved at the database level and remain visible to visitors; the seller is only prevented from *modifying* those premium fields going forward.

2.2.3 Product Management

FR-P1. A Seller shall be able to create, edit and delete products belonging to their own store(s).

FR-P2. Each product shall accept one or more images of supported MIME types, with a configurable maximum file size.

FR-P3. Each product shall support an arbitrary number of tags used by the recommendation engine.

FR-P4. Each product shall carry an `AIReviewed` flag controlled by the AI verification pipeline and overridable by an Administrator.

FR-P5. Products shall be browsable by category and sub-category.

2.2.4 Order System

FR-O1. A Customer shall be able to add and remove items in a cart persisted on the client side.

FR-O2. Checkout shall create a single `Order` entity even when the cart spans multiple stores, with one `OrderItem` per product line.

FR-O3. Order placement shall trigger a confirmation email through Nodemailer.

FR-O4. Each Customer shall be able to consult their personal order history.

FR-O5. Each Seller shall be able to consult the orders that include at least one product from their store.

2.2.5 Admin Dashboard

FR-D1. The Administrator shall access a dashboard listing pending seller applications, recent activity and platform-wide statistics.

FR-D2. The Administrator shall be able to approve or reject seller applications, granting the Seller role on approval.

FR-D3. The Administrator shall be able to suspend, reactivate or delete any store, with cascading deletion of its products and orders.

FR-D4. The Administrator shall be able to override the `adminReviewed` verdict of any product or store.

2.2.6 AI Verification

FR-V1. Upon creation, every store and product shall be sent to an n8n workflow (`cheebo-ai-verify`) that calls a hosted large-language model.

FR-V2. The workflow shall return a decision (*approve*, *flag*, *reject*) together with a textual rationale, written back to the entity through a callback endpoint.

FR-V3. Auto-approved entities shall display an *AI-verified* badge to customers.

FR-V4. Flagged entities shall be listed in the moderation queue for administrative review.

2.2.7 Featured Content Curation

FR-C1. The Administrator shall be able to mark any store or product as *featured*.

FR-C2. Featured entities shall be promoted on the landing page.

FR-C3. In the absence of featured products for a given tag, the recommendation engine shall fall back to top-selling products of that tag.

2.3 Non-Functional Requirements

The non-functional requirements, summarised in Table 2.1, were prioritised by the professional supervisor during the requirements review and define the qualities the deliverable must exhibit beyond pure functionality.

Table 2.1: Non-functional requirements

Criterion	**Description**
Performance	The catalogue and store pages must render in under one second on a desktop connection of 10 Mbit/s; API endpoints under the median use case must respond in under 300 ms.
Security	All passwords are hashed by Better-Auth; sessions are stored in HTTP-only, `SameSite=Lax` cookies; uploads are MIME- and size-validated; every protected route is gated by an `AuthGuard` and the `@Roles()` decorator.
Scalability	The backend is stateless and horizontally scalable behind a load balancer; uploads are written to a Docker volume that can be migrated to object storage with no code change.
Availability	The production stack runs under Docker Compose with restart policies; CI/CD deployment is zero-downtime through rolling container replacement.
Usability	The frontend follows the accessible component patterns of Radix UI, is fully responsive, and complies with the WCAG 2.1 AA contrast guidelines for primary text. Public URLs are slug-based and human-readable, improving share-ability and search-engine indexing.
Maintainability	The codebase is split into clearly-bounded NestJS modules, the schema is versioned through Prisma migrations, and the infrastructure is reproducible through Terraform. Storefront theming is driven entirely by CSS variables on a single scoped wrapper, so a per-store re-skin requires no component-level changes.
Observability	Application logs are emitted to `stdout` and aggregated by Docker; the chat assistant offers a queryable view of the database for operational diagnostics.
Internationalisation	The interface is currently English-only; copy is centralised to enable future translation without code changes.

2.4 Use Case Diagrams

The use cases below were derived directly from the functional requirements in the previous section. They are presented from the most abstract (global) to the most granular (per actor), so that the reader can zoom from the platform-level overview into each role's specific interactions.

2.4.1 Global Use Case Diagram

Figure 2.1 groups the principal use cases of Cheebo around the three actors. Two dotted dependencies illustrate the cross-actor flows on which the rest of the chapter relies: a seller application *triggers* an administrative review (`«triggers»`), and product creation is automatically *verified by AI* (`«verified by AI»`).

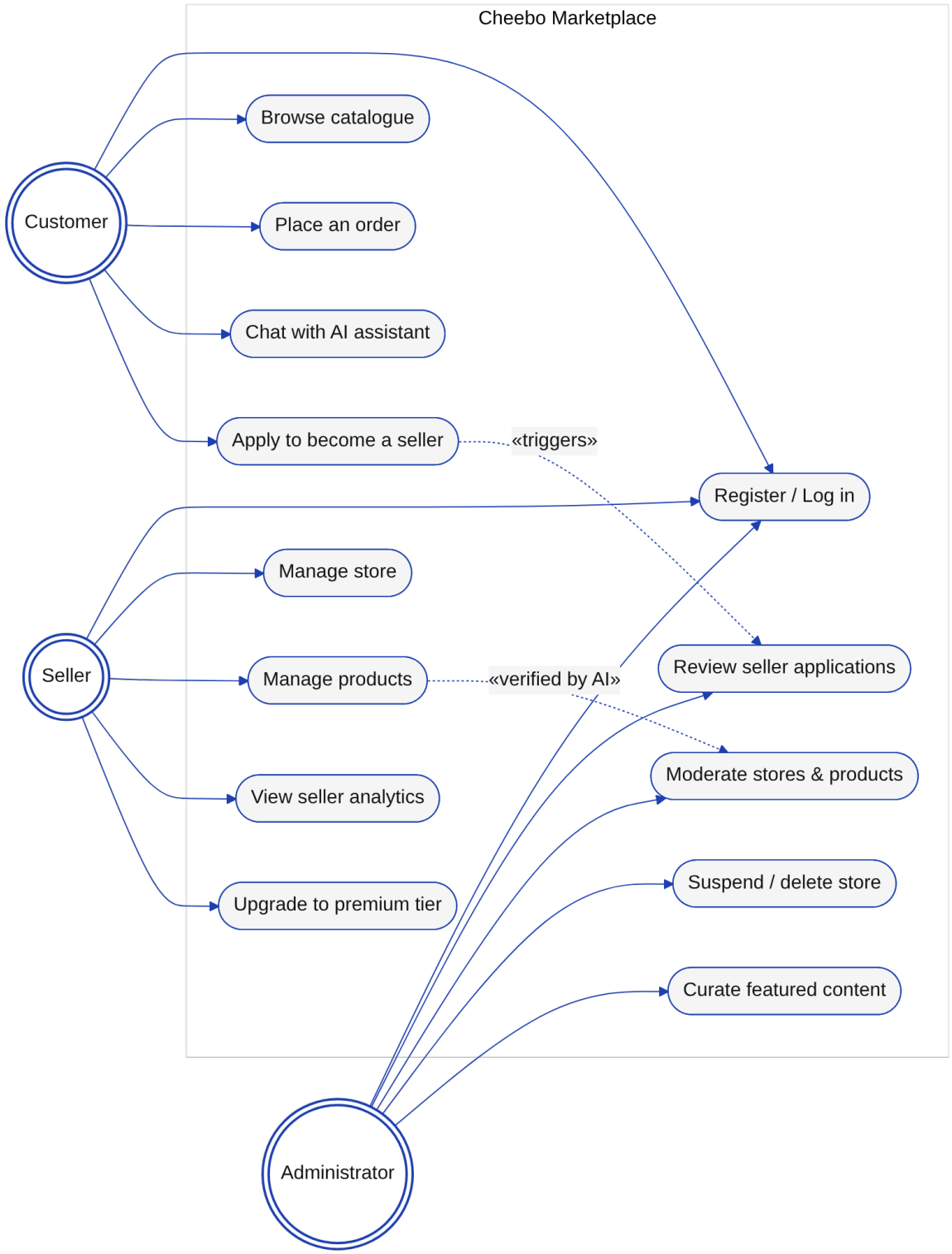

Figure 2.1: Global use case diagram

2.4.2 Detailed Use Cases — Customer

The Customer view in Figure 2.2 expands the shopping path from registration to order placement. Registration «includes» email verification, and checkout «includes» both order placement and an authenticated session.

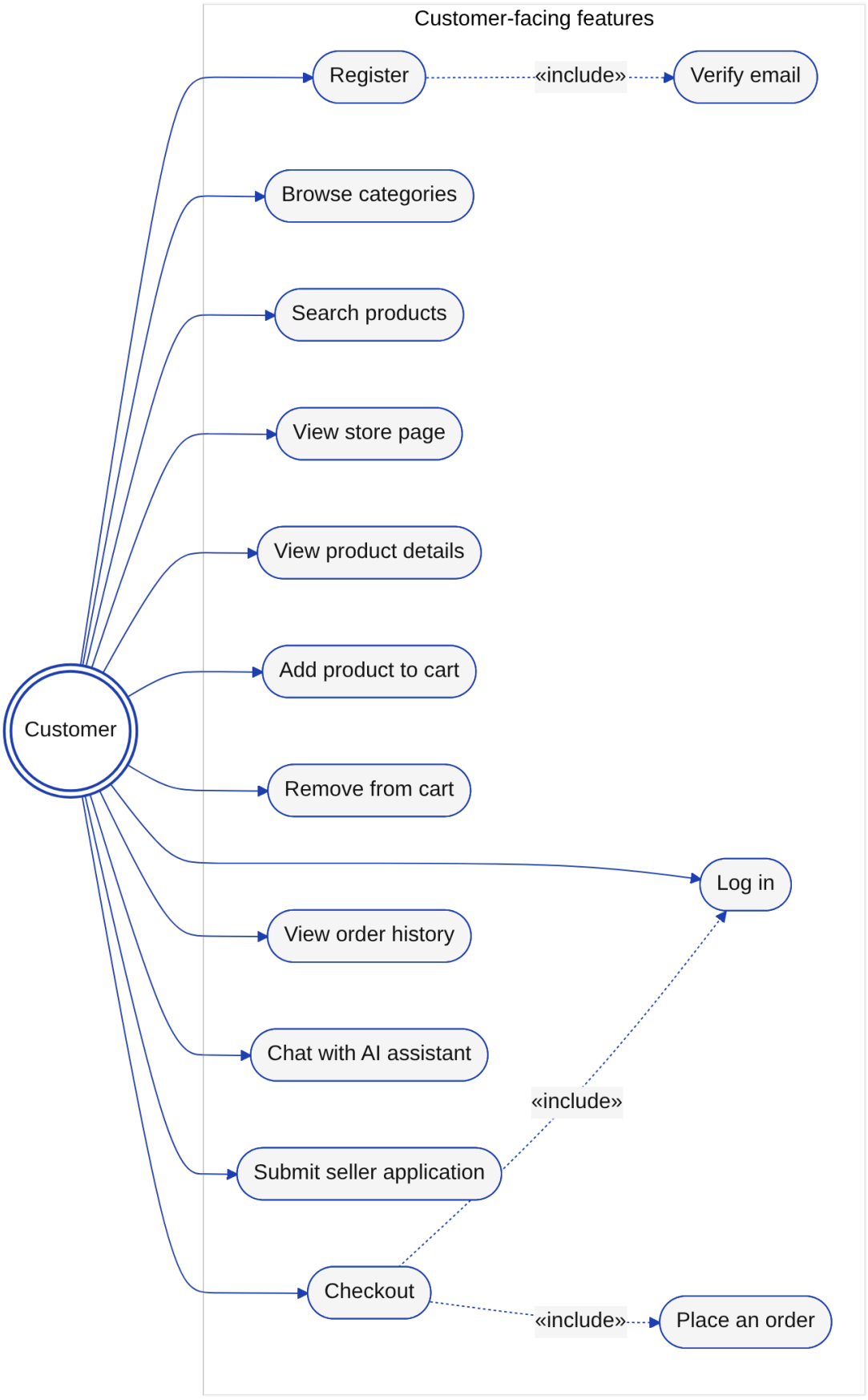

Figure 2.2: Customer use cases

2.4.3 Detailed Use Cases — Seller

Figure 2.3 shows that store creation depends on a prior approved seller application, and that opening additional stores depends on having activated the premium tier — the two gating conditions that govern the seller funnel.

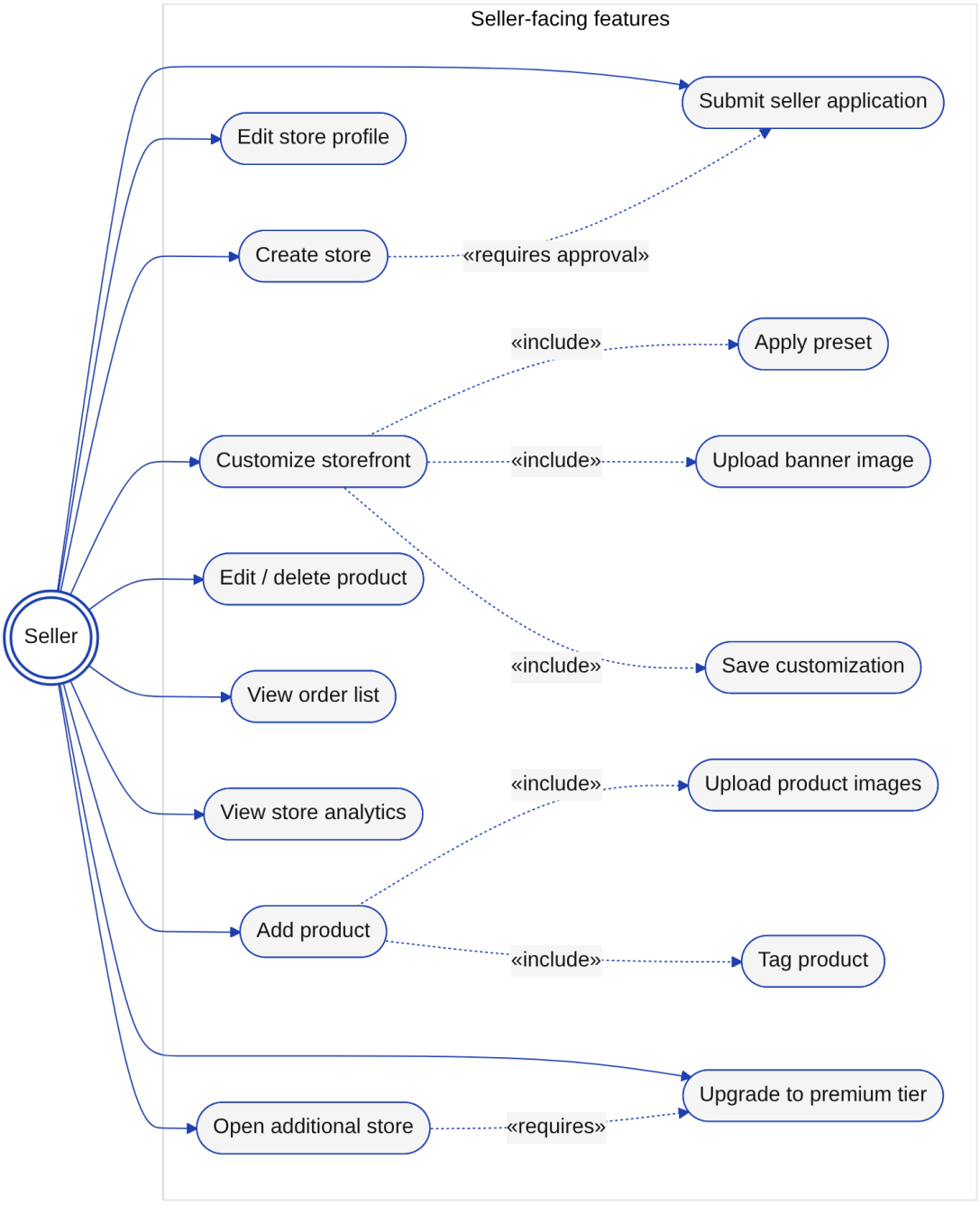

Figure 2.3: Seller use cases

2.4.4 Detailed Use Cases — Administrator

Figure 2.4 highlights the AI verification subsystem as a secondary actor that injects *auto-flag* events into the administration queue. The Administrator remains the final

authority on every override, suspension and deletion.

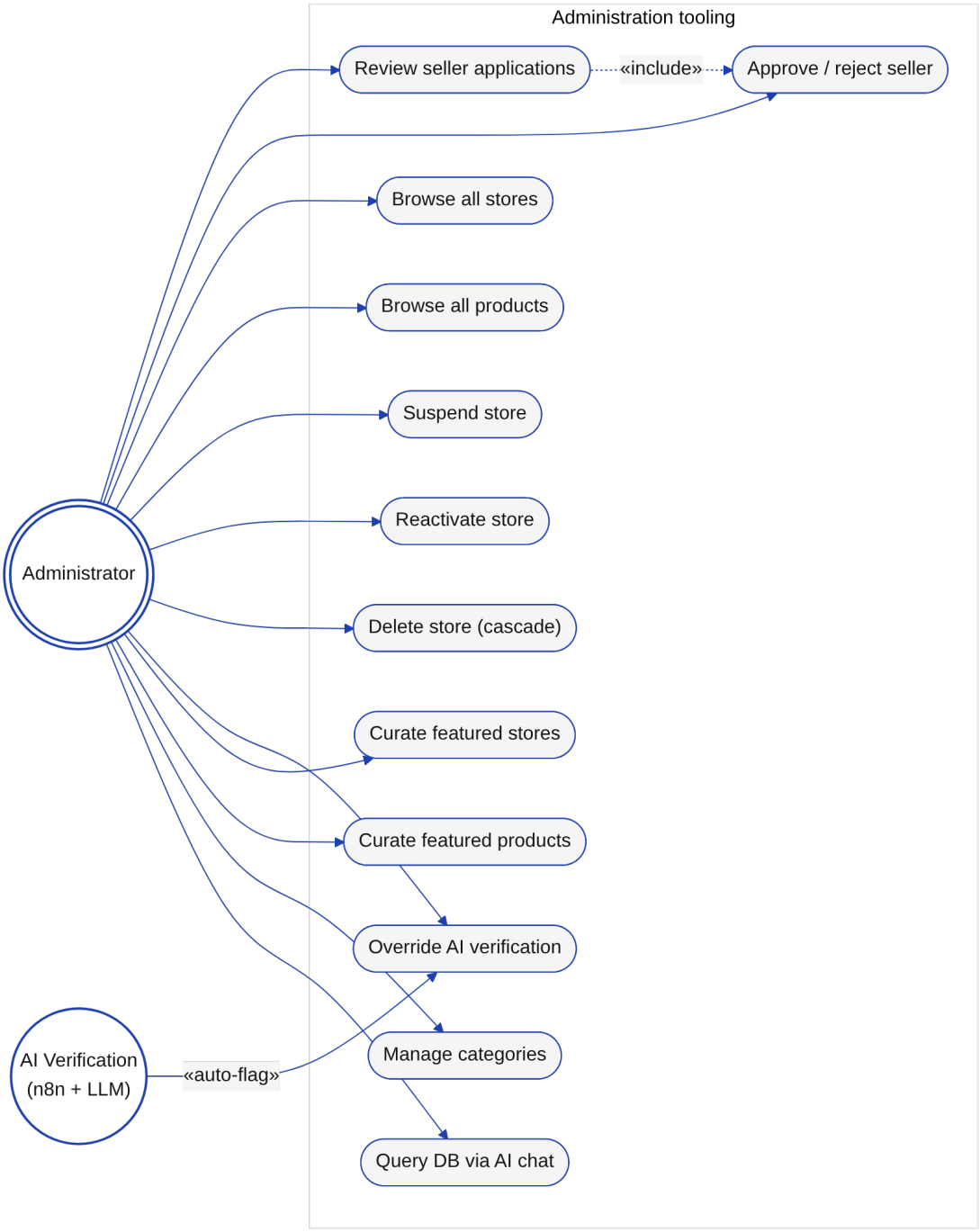

Figure 2.4: Administrator use cases

2.5 Sequence Diagrams

Three flows are particularly load-bearing for the architecture of the system: authentication, order placement, and AI-driven product verification. Each is described below with a UML sequence diagram generated from PlantUML sources stored under `diagrams/` in the project repository.

2.5.1 Registration, Email Verification and Login

Figure 2.5 traces the authentication flow. Registration creates a `User` row with `emailVerified=false` and triggers a Nodemailer message containing a one-time token; the user activates the account by clicking the link, which flips the flag and allows them to log in. Both Better-Auth and the NestJS layer participate, with sessions materialised as HTTP-only cookies returned by the API.

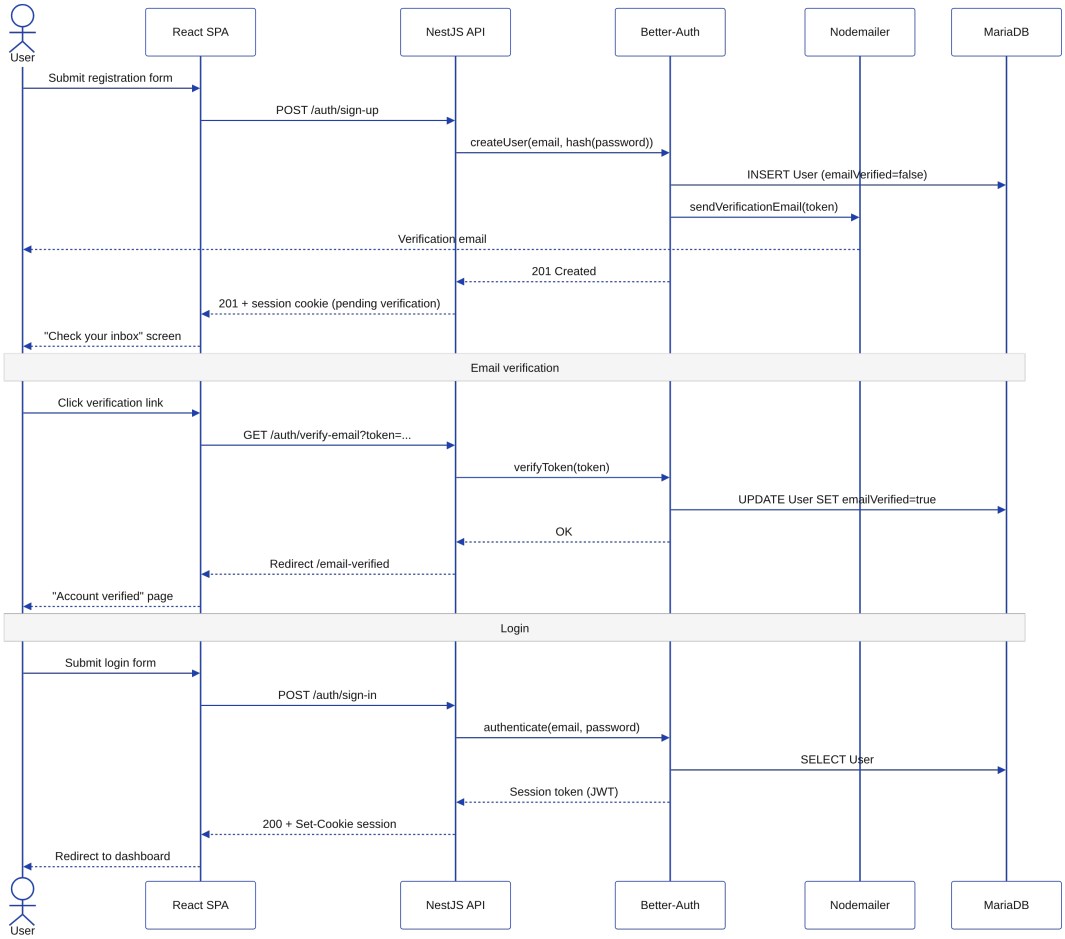

Figure 2.5: Sequence: registration, email verification and login

2.5.2 Order Placement

Figure 2.6 describes the checkout path. The cart is held client-side in a Zustand store and posted to the API as a single payload. The `AuthGuard` validates the session before the Order module persists the `Order` row and delegates the creation of line items to the OrderItems module; a confirmation email is sent before the response returns to the SPA, which then clears the cart store and redirects to the success page.

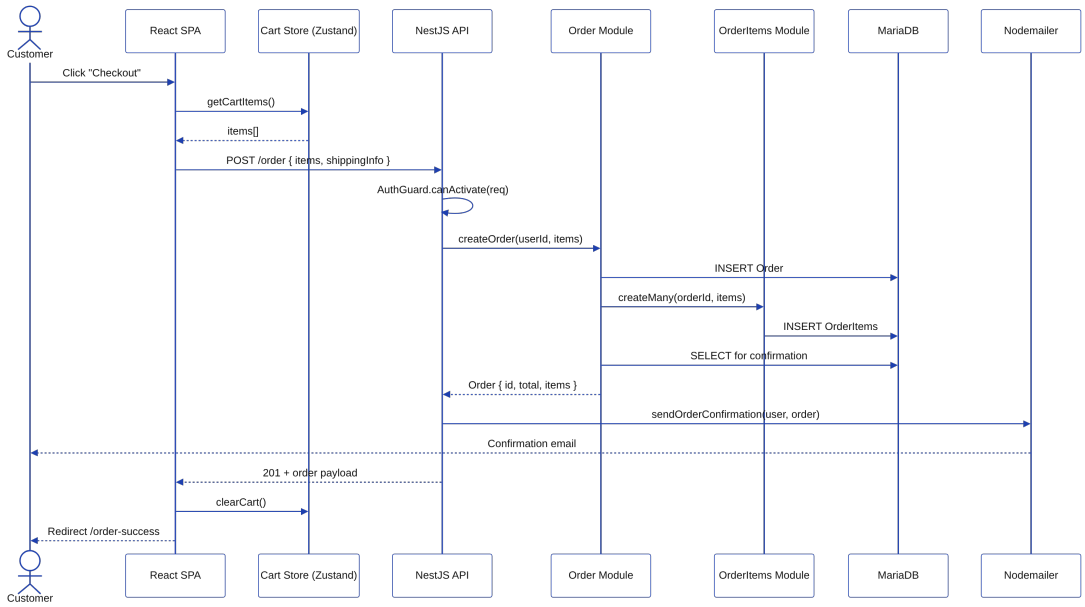

Figure 2.6: Sequence: order placement

2.5.3 Product Verification (AI + Administrator)

Figure 2.7 shows the dual-track verification process. Every product creation is forwarded to the `cheebo-ai-verify` webhook on n8n, which prompts a hosted LLM (Gemini or Groq) to apply the marketplace rules. The decision flows back to the backend through a callback endpoint and updates the `adminReviewed` column. The `alt` block at the bottom of the diagram captures the two outcomes: automatic approval (badge shown to the customer) and flagging (entry in the moderation queue, manual override by the Administrator).

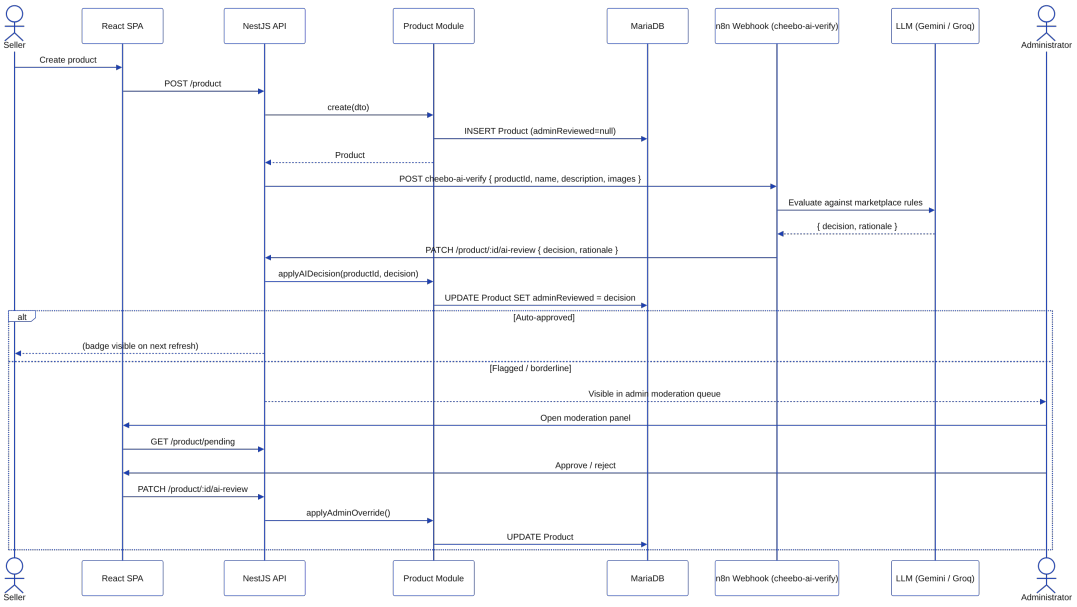

Figure 2.7: Sequence: product verification (AI and administrator)

Key Takeaways

- Three actors — Customer, Seller, Administrator — are modelled as roles on a single `User` table; an account can be promoted across roles without re-registering.
- Functional requirements are grouped into seven domains (authentication, store, product, order, admin dashboard, AI verification, featured curation), each mapped one-to-one with a backend NestJS module.
- The non-functional contract privileges *security*, *performance* and *maintainability* as first-order qualities, with concrete targets stated in Table 2.1.
- Three sequence diagrams illustrate the load-bearing flows: Better-Auth registration/login, multi-store order placement, and AI-then-administrator product verification.

Chapter 3 Design and Architecture

This chapter presents the global and detailed system architecture, database design, API design, and the N8N automation workflow designs.

3.1 Global Architecture

3.1.1 3-Tier Architecture Overview

Figure 3.1: Global system architecture

3.1.2 N8N as an Automation Layer

3.2 Detailed Technical Architecture

3.2.1 NestJS Modular Architecture

[IMAGE TO INSERT: NestJS module diagram (Auth, Store, Product, Order, AI...)]

Figure 3.2: NestJS modular architecture

3.2.2 React SPA Architecture

3.2.3 Docker Compose Services Diagram

[IMAGE TO INSERT: Docker Compose services diagram (frontend, backend, db, n8n)]

Figure 3.3: Docker Compose services

3.3 Database Design

3.3.1 Entity-Relationship Diagram

Figure 3.4: Entity-Relationship diagram

3.3.2 Prisma Schema Overview

3.3.3 Explanation of Key Relationships

3.4 API Design

3.4.1 REST API Design Principles

3.4.2 Endpoint Structure

Table 3.1: Main REST API endpoints

Method	Route	Description
POST	/auth/sign-up	Register a new user
POST	/auth/sign-in	Log in
GET	/store	List stores
POST	/store	Create a store
GET	/product	List products
POST	/product	Create a product
GET	/order/my	Get my orders
POST	/seller-application	Submit a seller application

3.4.3 Authentication Flow (BetterAuth)

[IMAGE TO INSERT: Sequence diagram — BetterAuth flow (JWT + session)]

Figure 3.5: BetterAuth authentication flow

3.5 N8N Workflow Design

3.5.1 AI Verification Workflow

Figure 3.6: N8N workflow — AI verification

3.5.2 Database Chat Assistant Workflow

Figure 3.7: N8N workflow — Database chat assistant

3.6 Class Diagrams

Figure 3.8: Class diagram

Chapter 4 Development and Implementation

This chapter details the development environment, backend and frontend implementation decisions, and the AI integration via N8N.

4.1 Development Environment

4.1.1 Hardware and Software

4.1.2 VSCode and Extensions

4.1.3 pnpm Monorepo Structure

[IMAGE TO INSERT: Monorepo directory tree (backend/, frontend/, terraform/)]

Figure 4.1: Cheebo monorepo structure

4.2 Backend (NestJS)

4.2.1 Project Structure

4.2.2 Key Modules

4.2.3 Authentication Guards and Role-Based Access Control

[IMAGE TO INSERT: Code snippet — AuthGuard + @Roles() decorator]

Figure 4.2: Role-based access control implementation

4.2.4 File Upload System (Multer)

4.2.5 Email System (Nodemailer)

4.3 Frontend (React + Vite)

4.3.1 Project Structure

4.3.2 Routing and Layouts

4.3.3 State Management (TanStack Query + Zustand)

4.3.4 Admin Dashboard Panels

Figure 4.3: Admin dashboard

4.3.5 Store and Product Management Interface

[IMAGE TO INSERT: Screenshot — Store / product management (seller view)]

Figure 4.4: Store and product management interface

4.4 AI Integration with N8N

4.4.1 AI Product/Store Verification Pipeline

4.4.2 Database Chat Assistant

4.4.3 Google Gemini / Groq Integration

4.4.4 N8N Workflow Implementation

4.5 Interface Screenshots

4.5.1 Landing Page

Figure 4.5: Landing page

4.5.2 Store Dashboard

Figure 4.6: Store dashboard

4.5.3 AI Verification Panel

Figure 4.7: AI verification panel

4.5.4 Analytics Page

Figure 4.8: Analytics page

Chapter 5 Infrastructure and Deployment

This chapter covers Docker containerisation, infrastructure as code with Terraform, the GitHub Actions CI/CD pipeline, DNS management, and the production environment.

5.1 Containerisation with Docker

5.1.1 Dockerfiles (Frontend and Backend)

5.1.2 Docker Compose Configuration

5.1.3 Multi-Service Networking

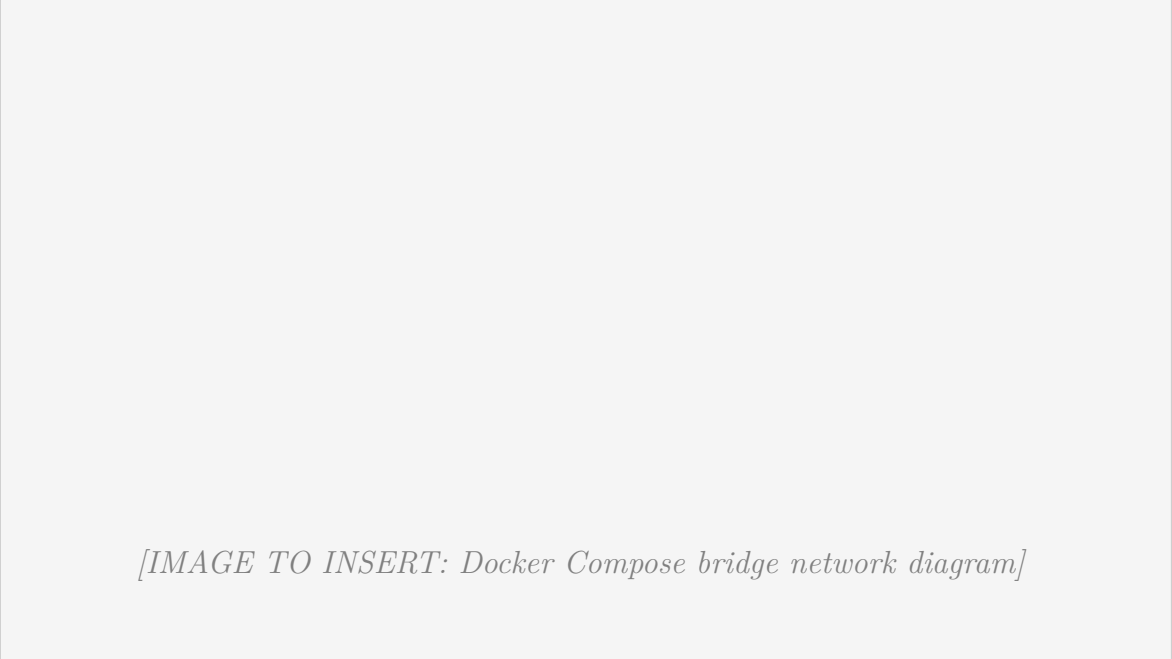

Figure 5.1: Docker Compose network architecture

5.2 Infrastructure as Code with Terraform

5.2.1 DigitalOcean Provider

5.2.2 VPC, Droplet and Firewall Resources

5.2.3 DNS Management with DigitalOcean

Rather than managing DNS records at the domain registrar (name.com) through a separate API, the project delegates DNS entirely to DigitalOcean using the same Terraform provider that manages the droplet. Two resources are declared in `main.tf`:

- `digitalocean_domain` — creates the DNS zone for `nadimjebali.engineer` on DigitalOcean's authoritative nameservers and automatically adds the apex (`@`) A record pointing to the droplet's IPv4 address.
- `digitalocean_record` — adds a second A record for the `www` subdomain, also pointing to the droplet.

Both records reference `digitalocean_droplet.cheebo.ipv4_address` directly. This means that if the droplet is ever replaced (e.g. after a destructive Terraform change), Terraform updates both DNS records in the same `apply` run that creates the new droplet, with no manual intervention required.

The domain's nameservers are delegated to DigitalOcean (`ns1-ns3.digitalocean.com`) from the name.com registrar panel, which is a one-time configuration step. After that, A-record updates propagate in under five minutes thanks to a 300-second TTL.

> **Key Takeaways**
>
> DNS is version-controlled alongside infrastructure — the domain always points to the current droplet IP with no manual update step.
>
> TTL 300 s means a full redeploy (destroy old droplet, create new one, update DNS) converges in under ten minutes end-to-end.

5.2.4 Remote State Management with DigitalOcean Spaces

Figure 5.2: Terraform resources

5.3 CI/CD Pipeline with GitHub Actions

5.3.1 Workflow Overview

Figure 5.3: GitHub Actions CI/CD pipeline

5.3.2 Build Stage (Docker Build and Push to Docker Hub)

5.3.3 Deployment Stage (SSH and Docker Compose)

5.3.4 Environment Secrets Management

5.4 Production Environment

5.4.1 DigitalOcean Droplet Specifications

The production workload runs on a single DigitalOcean droplet provisioned by Terraform with the following specifications:

Property	**Value**
Region	Frankfurt (fra1)
Size	s-2vcpu-2gb (2 vCPUs, 2 GB RAM)
Image	Ubuntu 24.04 LTS
VPC	cheebo-vpc (192.168.1.0/24)
Public URL	`http://nadimjebali.engineer`
DNS	Managed by DigitalOcean (Terraform)

The application is accessed exclusively through the domain name `nadimjabali.engineer`. The raw droplet IP is reserved for SSH access by the CI/CD pipeline and is never exposed to end users. CORS on the backend is configured with the domain as the single trusted origin, so any attempt to reach the API directly from the IP is rejected by the browser's same-origin policy.

5.4.2 Production Service Architecture

[IMAGE TO INSERT: Production environment diagram (Droplet + services)]

Figure 5.4: Production architecture

5.4.3 N8N Deployment Alongside the Application

General Conclusion

Summary of Achievements

Implemented Features vs Initially Planned Features

Challenges Encountered and Solutions Provided

Current Implementation Limitations

Future Improvements

Bibliography and Webography

- NestJS Official Documentation — https://docs.nestjs.com
- React Official Documentation — https://react.dev
- Prisma ORM Documentation — https://www.prisma.io/docs
- TanStack Query Documentation — https://tanstack.com/query
- Docker Documentation — https://docs.docker.com
- Terraform Documentation — https://developer.hashicorp.com/terraform
- N8N Documentation — https://docs.n8n.io
- GitHub Actions Documentation — https://docs.github.com/en/actions
- DigitalOcean Developer Docs — https://docs.digitalocean.com

Chapter A Complete Prisma Schema

Listing A.1: Prisma schema — excerpt

```
% TODO: paste the contents of backend/prisma/schema/ here
```

Chapter B API Endpoints Reference

Chapter C N8N Workflow JSON Exports

Listing C.1: N8N workflow — AI verification (JSON)

```
% TODO: paste the JSON exported from N8N
```

Chapter D Docker Compose File

Listing D.1: docker-compose.yml

```
% TODO: paste the contents of docker-compose.yml
```

Chapter E GitHub Actions Workflow

Listing E.1: .github/workflows/deploy.yml

```
% TODO: paste the CI/CD workflow content
```